

Question 1 - Movie Ratings (Lists and Strings)

Approach

- The movie data is first split into separate entries using commas.
- Each entry is separated into a movie title and rating using a colon.
- `strip()` removes extra spaces and the rating is converted to int.
- `average_rating()` collects all scores for the requested title and rounds the mean to one decimal place.
- `best_movie()` compares the averages of the distinct movie titles.
- `rating_counts()` stores the number of ratings for each movie in a dictionary.

Demo Output

```
Parsed ratings:
[('Dune', 8), ('Dune', 9), ('Barbie', 7),
 ('Dune', 10), ('Barbie', 9), ('Oppenheimer', 9),
 ('Barbie', 6)]
```

```
Average ratings:
Dune: 9.0
Barbie: 7.3
Oppenheimer: 9.0
Unknown: 0.0
```

```
Best movie:
Dune
```

```
Rating counts:
{'Dune': 3, 'Barbie': 3, 'Oppenheimer': 1}
```

Issues Faced / Checks

- Extra whitespace must not change the movie title.
- An unknown title must return 0.0 instead of causing division by zero.
- Dune and Oppenheimer tie at 9.0; Dune is returned because it is encountered first.

Question 2 - Gradebook (Dictionaries and Sets)

Approach

- The gradebook uses a dictionary of subjects containing student-grade dictionaries.
- `subjects_of()` adds matching subjects to a set.
- `takes_all()` uses set intersection to find students present in every subject.
- `student_average()` calculates the mean of all grades belonging to the requested student.
- `honor_roll()` keeps students whose average is at most the supplied limit and sorts the names alphabetically.

Demo Output

```
Subjects of anna: {'math', 'art'}  
Students taking all subjects: set()  
Average of clara: 1.15  
Average of unknown: 0.0  
Honor roll: ['clara']
```

Issues Faced / Checks

- Set intersection is used rather than counting subjects.
- Missing students return 0.0.
- The honor-roll boundary is inclusive because the condition is `<= limit`.
- The final list is alphabetically sorted.

Question 3 - Recipe Scaler (Functions)

Approach

- `scale_recipe()` accepts an arbitrary number of ingredient tuples with `*ingredients`.
- Every amount is multiplied by the requested serving count.
- The scaled amounts are returned in a dictionary and printed with the selected unit.
- Keyword arguments are displayed as cooking notes.
- Servings below 1 are rejected and the function returns 0.

Demo Output

```
Shopping list: Pancakes
Servings: 2
flour: 200g
milk: 300g
sugar: 40g
```

```
Shopping list: Fresh Juice
Servings: 3
orange juice: 600ml
water: 300ml
```

```
Shopping list: Chocolate Cake
Servings: 4
flour: 400g
sugar: 200g
butter: 160g
Cooking notes:
oven: 180C
time: 45min
```

```
Error: servings must be at least 1 for Invalid Recipe.
```

Issues Faced / Checks

- The ingredient count is variable, so `*ingredients` is required.
- The unit is configurable because one demonstration uses ml.
- `**options` allows several cooking notes.
- The invalid-serving case must return exactly 0.

Question 4 - Library (Classes and Inheritance)

Approach

- Book contains title, author, and year and implements `__str__`, `__eq__`, and `age()`.
- Equality compares title and author only.
- EBook inherits Book, adds `size_mb`, overrides `__str__`, and calculates download time.
- Library stores books in a list and ignores an equal book already present.
- `find_by_author()`, `oldest()`, and `__len__()` implement the remaining library operations.

Demo Output

```
Library length: 5
```

```
Books:
```

```
1984 by George Orwell (1949)
```

```
Dune by Frank Herbert (1965)
```

```
The Hobbit by J.R.R. Tolkien (1937)
```

```
Clean Code by Robert C. Martin (2008) - 5.0 MB
```

```
Python Crash Course by Eric Matthes (2019) - 8.0 MB
```

```
Book age:
```

```
77
```

```
Equality:
```

```
True
```

```
Books by George Orwell:
```

```
1984 by George Orwell (1949)
```

```
Oldest book:
```

```
The Hobbit by J.R.R. Tolkien (1937)
```

```
EBook download time:
```

```
4.0 seconds
```

Issues Faced / Checks

- A duplicate with the same title and author is ignored even if its year differs.
- EBook reuses the Book implementation through inheritance.
- Download time converts megabytes to megabits before dividing by network speed.
- The duplicate addition leaves the library length unchanged.